

Parallel programming models

Speedup

- Main motivation of parallel processing: achieve speedup

$$\text{speedup (using } P \text{ processors)} = \frac{\text{Execution time (1 processor)}}{\text{Execution time (} P \text{ processors)}}$$

Compute in parallel

- Sum of numbers in an array
- `Nums=[1,2,3,4,5,6,7,8,9,10]`
- Master-slave model & message passing
- $P=5$ processors (students)
- $P=3$ processors
- $P=1$ processor
- Analyze execution times

Issues of parallel programming

- Observations
 - Communication time limits speedup
 - Imbalance in work assignment limits speedup
- Goals for designing parallel programs
 - Decomposing work into pieces that can safely be performed in parallel
 - Assigning work to processors
 - Managing communication/synchronization between the processors so that it does not limit speedup

Programming model

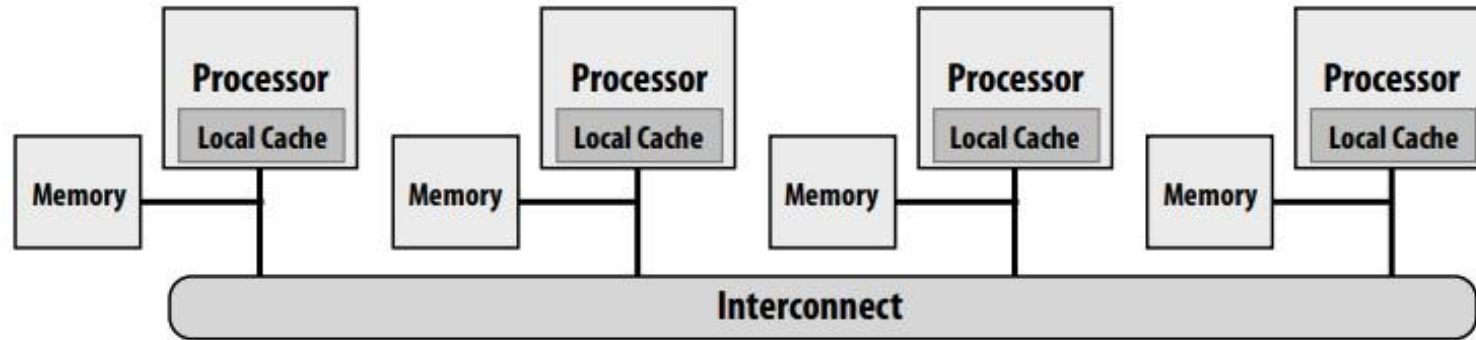
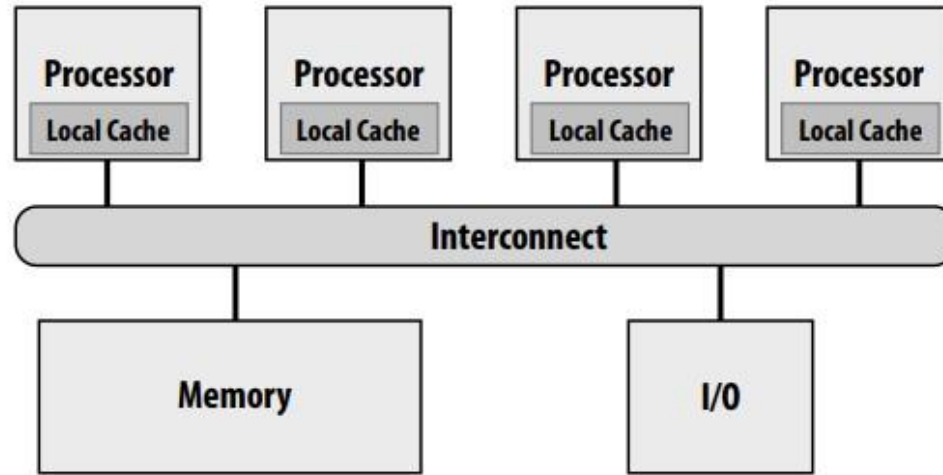
- Provides way of thinking about the structure of programs
- Parallel programs:
 - Abstractions for describing concurrent, parallel, or independent computation: threads, specific language constructions
 - Abstractions for describing communication
 - Shared address space
 - Message passing
 - Data parallel

Shared address space model

- Threads communicate reading/writing to shared variables
- Ensure mutual exclusion via use of locks
- Natural extension of sequential programming
- HW implementation:
 - UMA - uniform memory access time
 - NUMA- non-uniform memory access time
 - COMA – cache only memory architecture

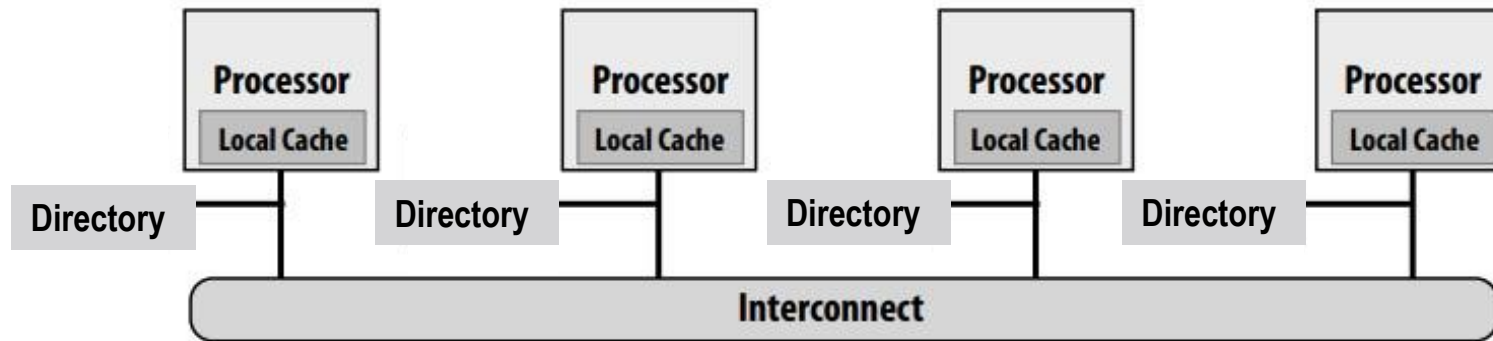
UMA, NUMA

UMA



NUMA

COMA

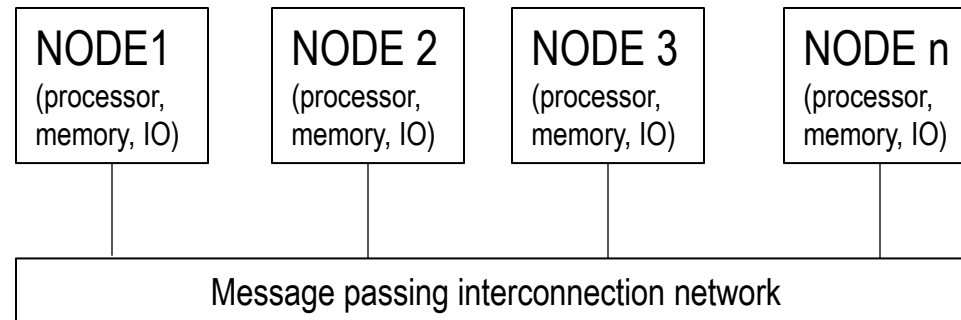


Message passing model

- Threads operate within their own private address spaces
- Threads communicate by sending/receiving messages
 - send: recipient, buffer to be transmitted and message ID
 - receive: sender, buffer to store data and message ID
- HW implementation: computing nodes able to communicate through messages

No-Remote Memory Access

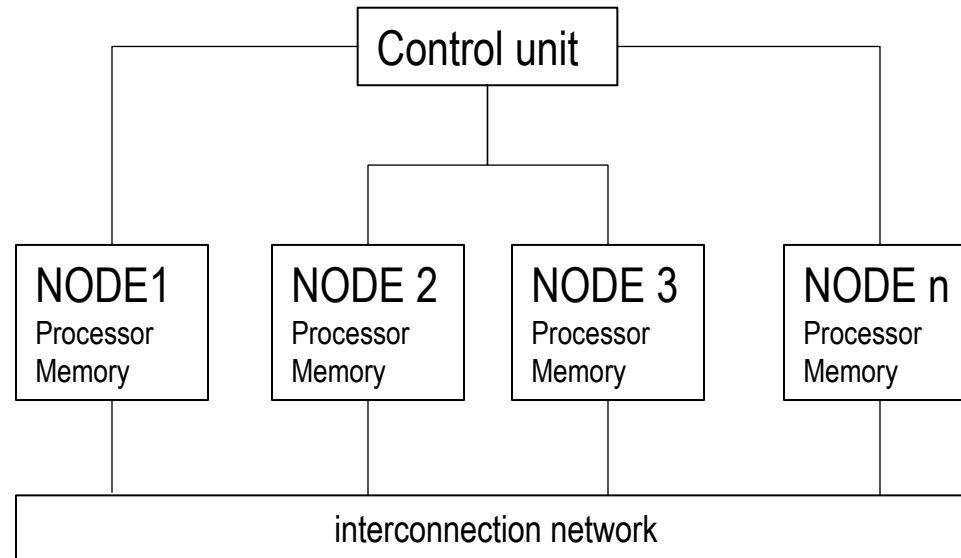
NORMA



Data parallel model

- Programs perform same operation on different data elements in a collection (ex: MAP - same function on each element of an array)
- Assumes shared address space
- Limits communication between iterations
- Stream programming: streams and kernels
- Communication: scatter/gather instructions

SIMD computers



To summarize

- Describe concurrent, parallel computations through:
 - Threads
 - Specific language constructions
- Programming models
 - Threads use shared variables
 - Threads/processes communicate through messages
 - Structure computation as a map over a collection of data, limit communication